

Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

CSE 4802: Software Maintenance Lab

Lab 4

Reverse Engineering and Code Reengineering

Using IDA Free, Doxygen, and IntelliJ IDEA

Antara Arifa Mullick

Student ID: 210042132

Program: B.Sc. in SWE
Department of CSE

Task Author: Lutfun Nahar Lota
CSE, IUT

1 Work Done

This lab was completed in two parts.

In the first part, a simple C program was compiled into a Windows executable and analyzed using IDA Free. The important functions were identified, their disassembly was examined, and the cross-references from `main()` to `add()` and `isEven()` were checked.

In the second part, a small Java Library Management application was created and executed in IntelliJ IDEA. Doxygen was then configured to recover documentation from the Java source code. Graphviz was installed so that Doxygen could generate diagrams.

This report only includes the work that was actually completed. The separate Java refactoring/reengineering stage is not included.

2 Tools and Environment

Table 1: Tools used in the lab

Tool	Version	Use
IDA Free	9.3.260421	Disassembled and analyzed the C executable.
GCC	16.1.0	Compiled the C source file into an executable.
Java JDK	21	Ran the Java application.
IntelliJ IDEA	Free edition	Created and executed the Java project.
Doxygen	1.17.0	Generated documentation from Java source code.
Graphviz	15.1.0	Supported diagram generation in Doxygen.

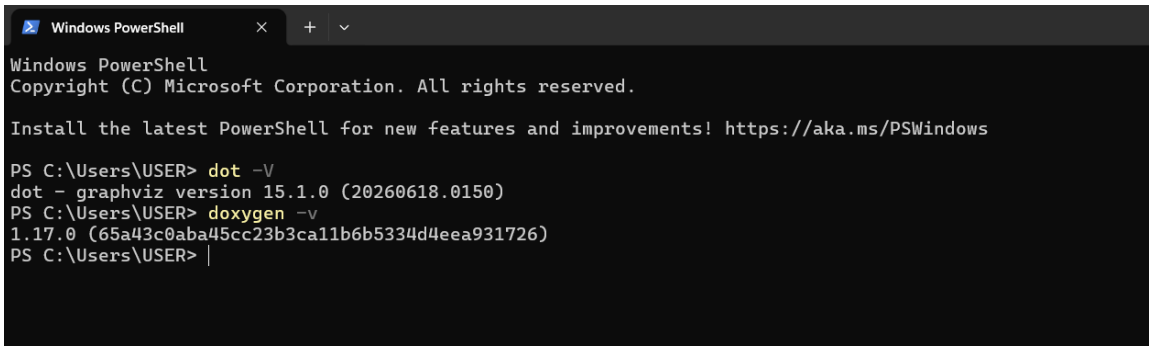


Figure 1: Doxygen and Graphviz version verification.

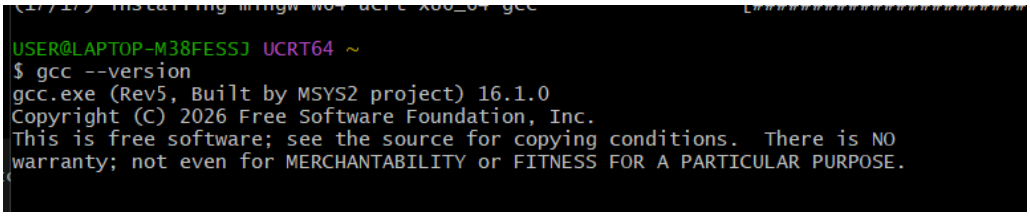


Figure 2: GCC version verification in MSYS2 UCRT64.

3 Part A: Reverse Engineering Using IDA Free

3.1 Installation of IDA Free

IDA Free provides a lot of useful features.

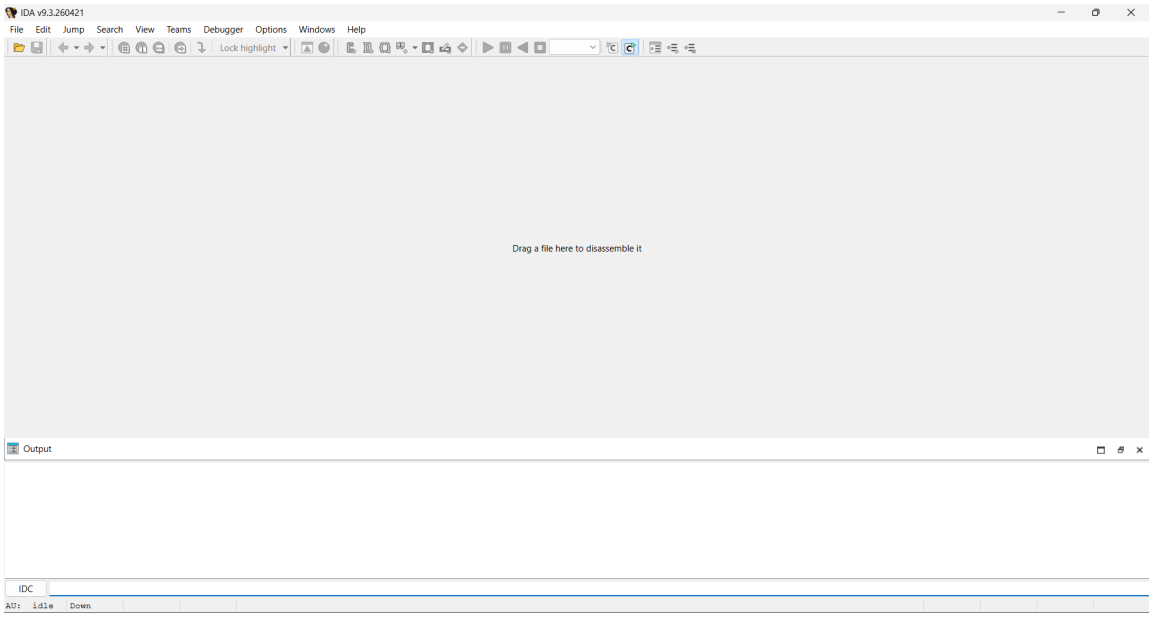


Figure 3: IDA Free opened successfully.

3.2 C Program and Compilation

A simple C program was used. It contained three important functions: `main()`, `add()`, and `isEven()`.

The program was compiled using GCC with the command:

```
gcc -O0 -g simple_calc.c -o simple_calc.exe
```

The program ran successfully and printed:

```
The result is even:  30
```

The source code and terminal output are shown in the following screenshots.

```
USER@LAPTOP-M38FESSJ MSYS ~
$ mkdir -p "/d/4.2/Maintenance lab/Lab4/IDA_Task"
cd "/d/4.2/Maintenance lab/Lab4/IDA_Task"

USER@LAPTOP-M38FESSJ MSYS /d/4.2/Maintenance lab/Lab4/IDA_Task
$
$ cat > simple_calc.c <<'EOF'
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int isEven(int number) {
    if (number % 2 == 0) {
        return 1;
    } else {
        return 0;
    }
}

int main() {
    int x = 10;
    int y = 20;
    int result = add(x, y);

    if (isEven(result)) {
        printf("The result is even: %d\n", result);
    } else {
        printf("The result is odd: %d\n", result);
    }

    return 0;
}
EOF

USER@LAPTOP-M38FESSJ MSYS /d/4.2/Maintenance lab/Lab4/IDA_Task
$ ls
simple_calc.c

USER@LAPTOP-M38FESSJ MSYS /d/4.2/Maintenance lab/Lab4/IDA_Task
$
```

Figure 4: Simple C program created for reverse engineering.

```
USER@LAPTOP-M38FESSJ UCRT64 /d/4.2/Maintenance lab/Lab4/IDA_Task
$ gcc -O0 -g simple_calc.c -o simple_calc.exe

USER@LAPTOP-M38FESSJ UCRT64 /d/4.2/Maintenance lab/Lab4/IDA_Task
$ ls
simple_calc.c  simple_calc.exe

USER@LAPTOP-M38FESSJ UCRT64 /d/4.2/Maintenance lab/Lab4/IDA_Task
$ ./simple_calc.exe
The result is even: 30

USER@LAPTOP-M38FESSJ UCRT64 /d/4.2/Maintenance lab/Lab4/IDA_Task
$
```

Figure 5: Compilation and execution of the C program.

3.3 Loading the Executable into IDA Free

The generated file `simple_calc.exe` was opened in IDA Free. IDA automatically analyzed the executable and displayed its functions and assembly instructions.

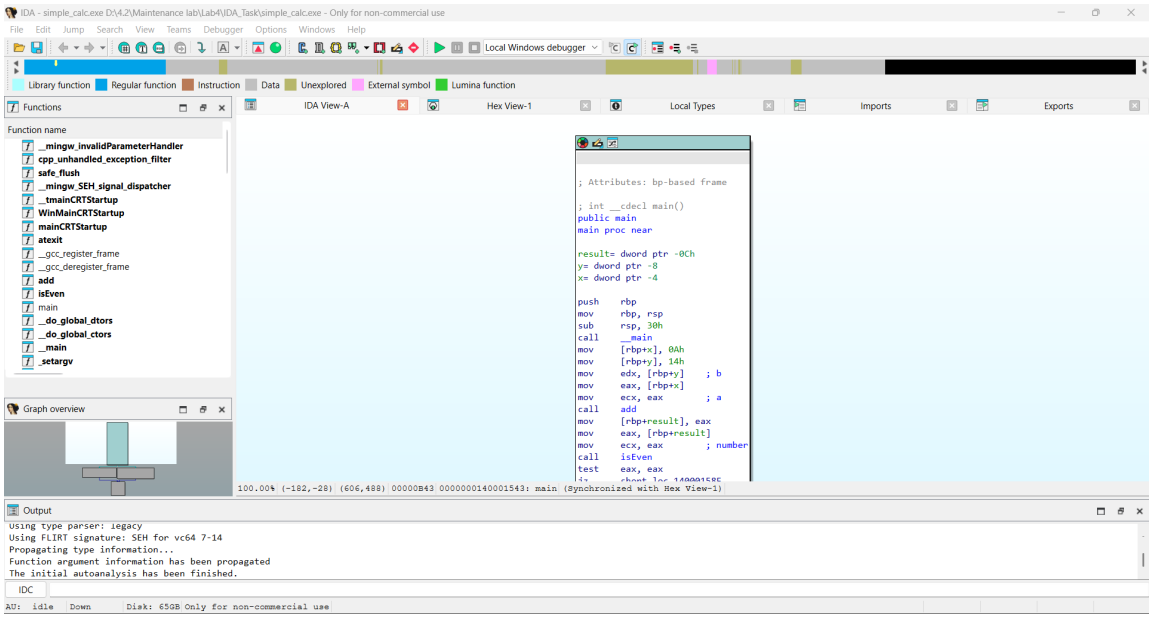


Figure 6: The executable loaded into IDA Free.

3.4 Functions Identified

IDA identified many runtime and compiler-generated functions. The three important user-defined functions were:

- `main()` – controls the complete program.
- `add()` – adds two integer values.
- `isEven()` – checks whether the result is even.

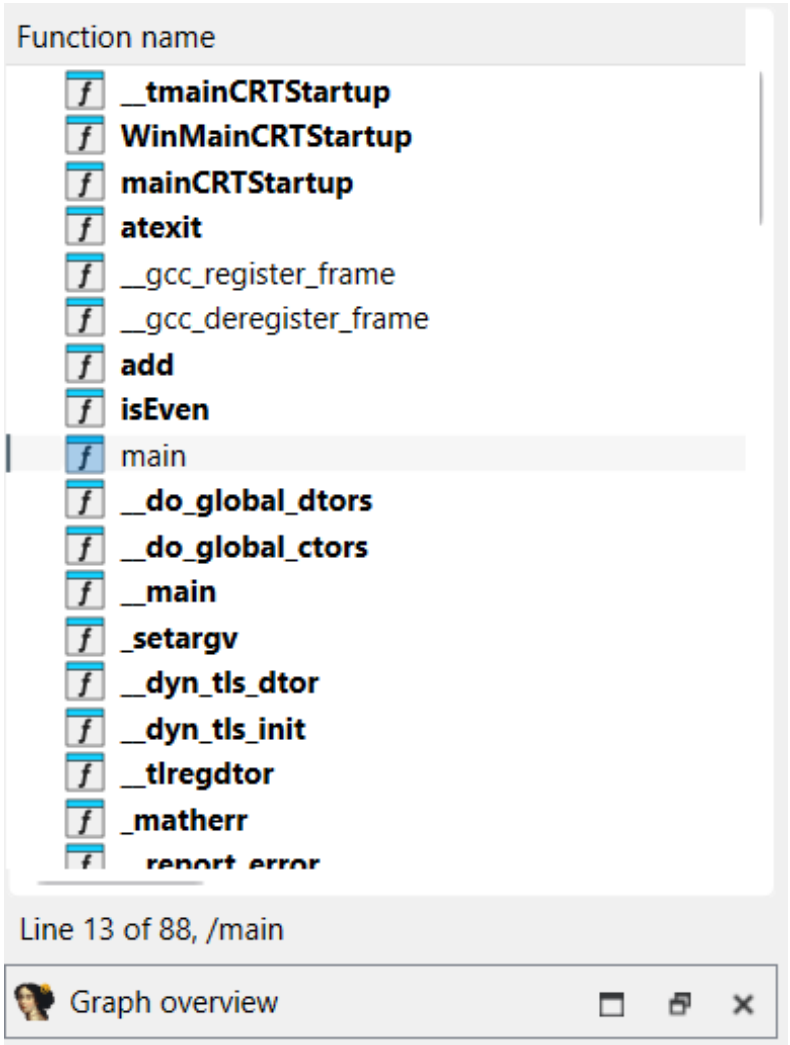


Figure 7: Important functions identified by IDA Free.

3.5 Disassembly of the `add()` Function

The `add()` function was very small. IDA showed that the two input values were loaded, added, and returned through the `EAX` register.

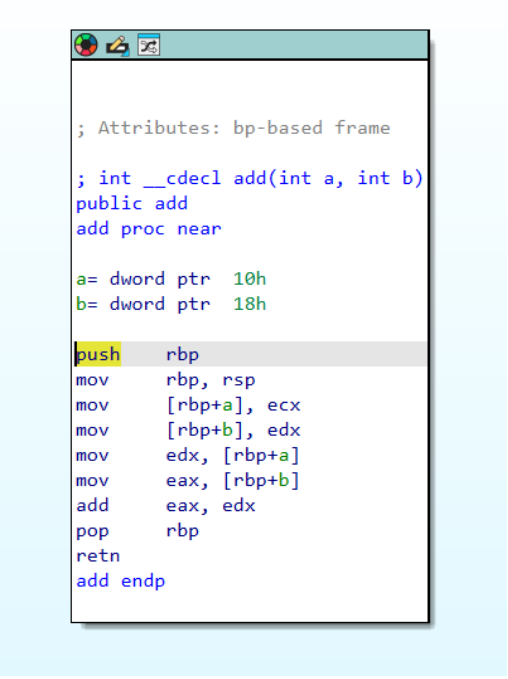


Figure 8: Disassembly output of the `add()` function.

3.6 Disassembly of the `isEven()` Function

The `isEven()` function checked the last bit of the number. If the last bit was 0, the number was even and the function returned 1. Otherwise, it returned 0.

The graph view clearly showed the two possible branches.

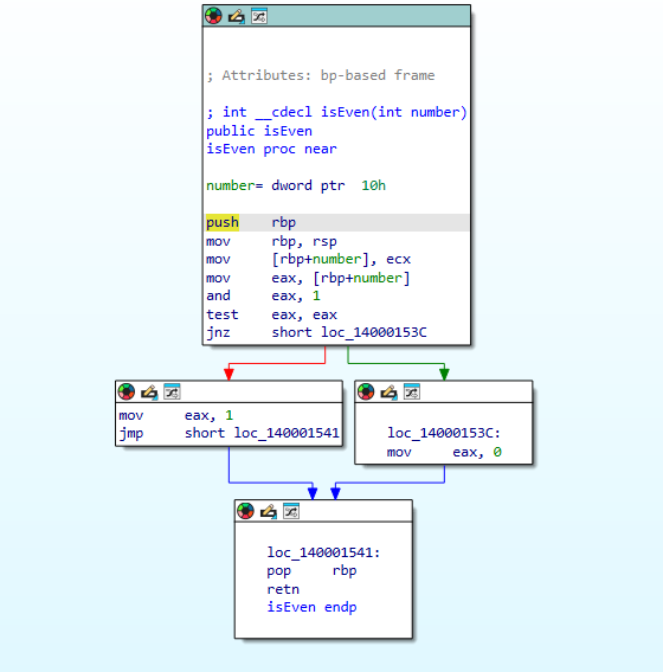


Figure 9: Disassembly output of the `isEven()` function.

3.7 Disassembly of the `main()` Function

The `main()` function stored the values 10 and 20, called `add()`, and stored the result. It then called `isEven()`.

Based on the returned value, the program selected either the even message or the odd message and called `printf()`.

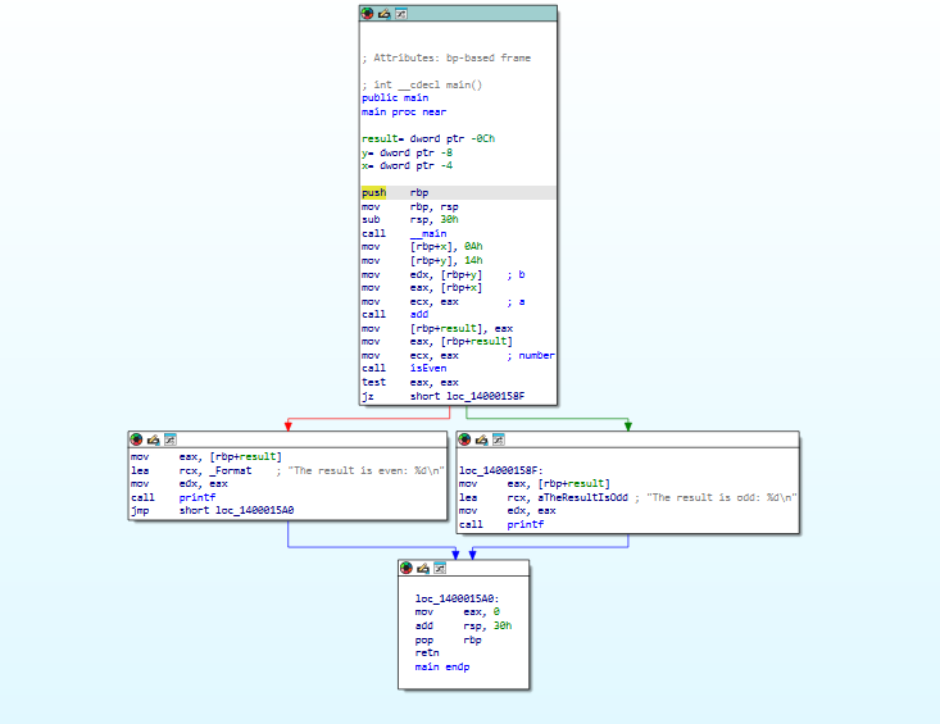


Figure 10: Disassembly output of the `main()` function.

3.8 Cross-Reference of `add()`

The cross-reference window showed:

```
main+23    call add
```

This means that the `add()` function is called from `main()`.

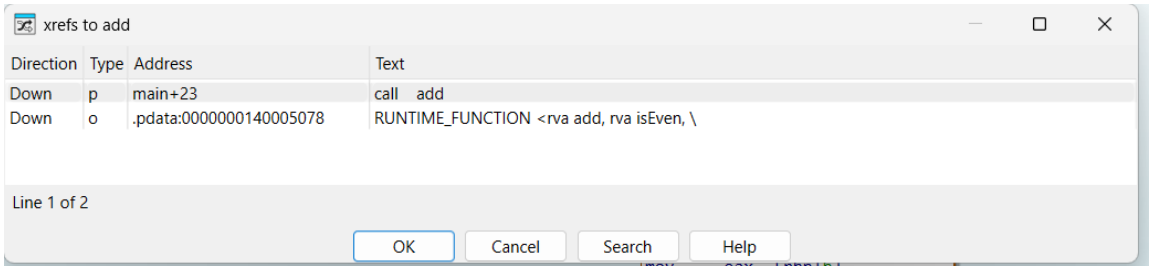


Figure 11: Cross-reference showing that `add()` is called from `main()`.

3.9 Cross-Reference of `isEven()`

The cross-reference window showed:

```
main+30    call isEven
```

This means that the `isEven()` function is also called from `main()`.

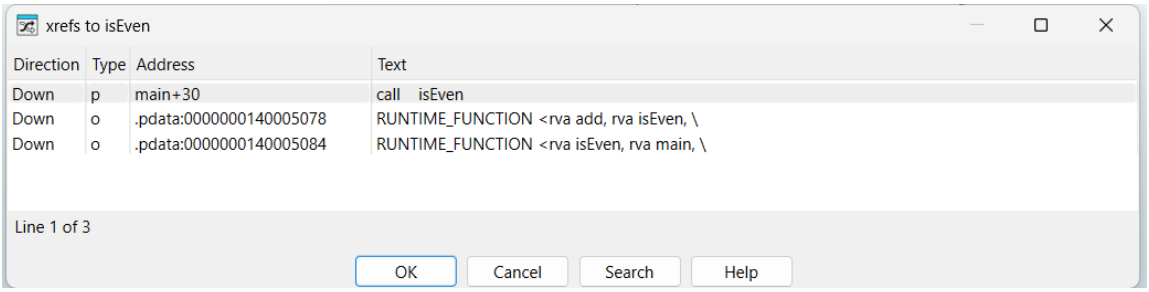


Figure 12: Cross-reference showing that `isEven()` is called from `main()`.

3.10 Result of IDA Analysis

IDA Free successfully recovered the main structure of the compiled program. From the executable, it was possible to understand that two values were added, the result was checked, and one of two messages was printed.

This shows how IDA can help maintainers understand a program even when they only have the compiled executable.

4 Part B: Reverse Engineering a Legacy Java Application Using Doxygen and IntelliJ IDEA

4.1 Selected Java Codebase

A small Library Management application was used as the Java codebase. It contained:

- A `Book` class.
- A `LibrarySystem` class.
- Methods for adding, issuing, returning, searching, and displaying books.

The code was written and executed in IntelliJ IDEA.

4.2 Original Java Code

The complete Java source code is already visible in the screenshots, so it is not repeated in this report.

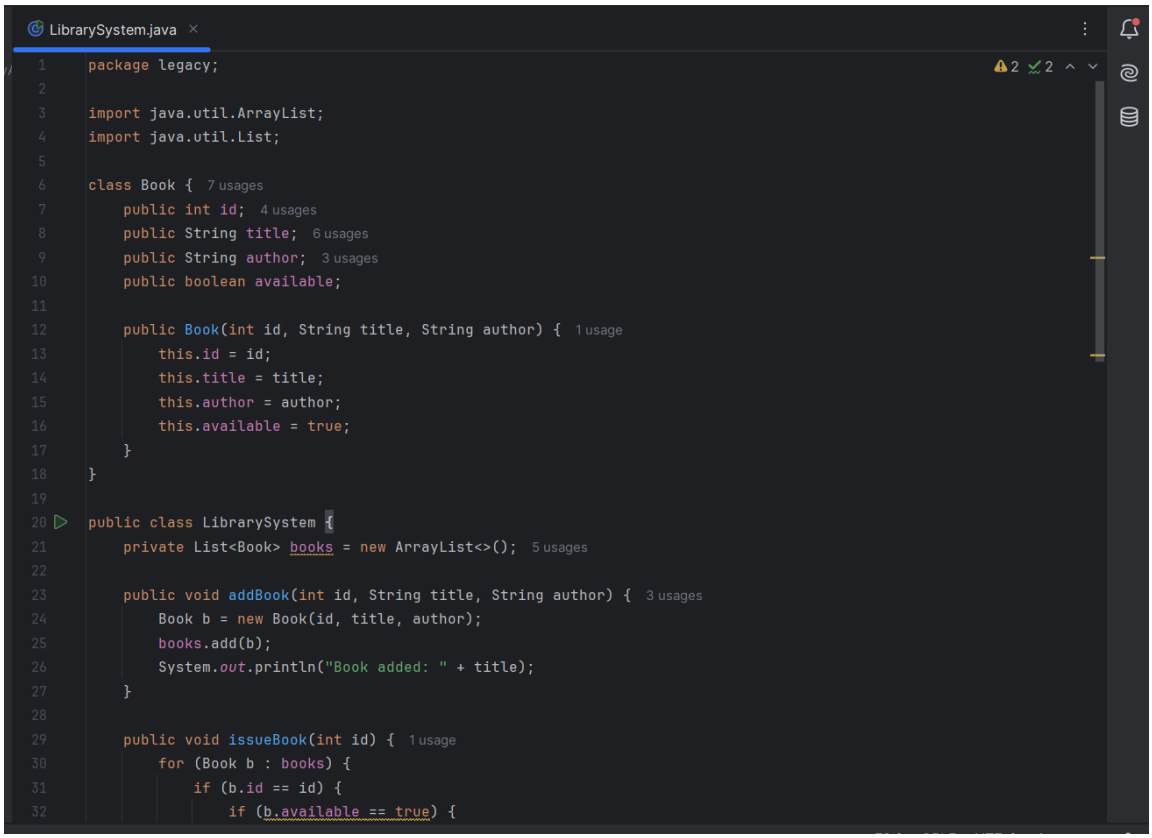


Figure 13: First part of the Java Library Management code.



Figure 14: Middle part of the Java Library Management code.

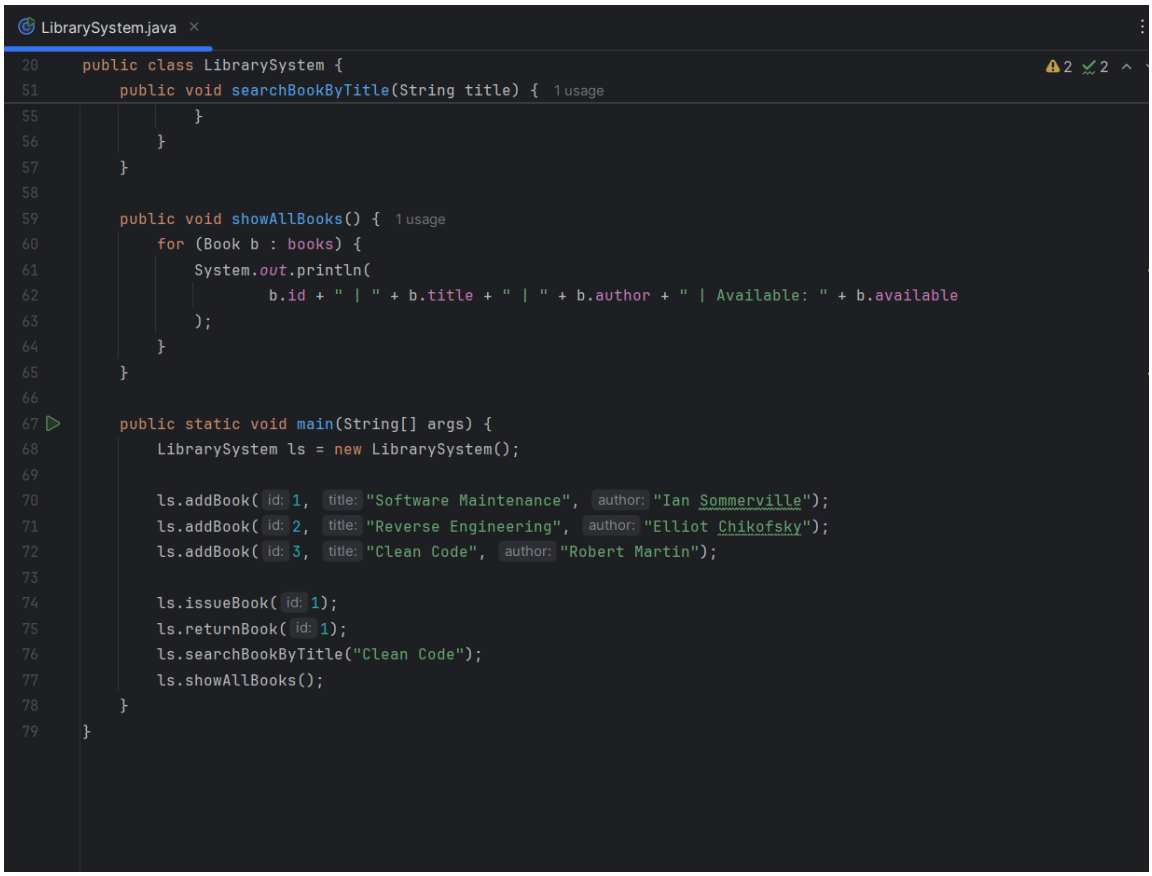


Figure 15: Final part of the Java Library Management code.

4.3 Output of the Java Application

The application successfully:

- Added three books.
- Issued the book *Software Maintenance*.
- Returned the same book.
- Found the book *Clean Code*.
- Displayed all books and their availability.

The process finished with exit code 0, showing that the application ran successfully.

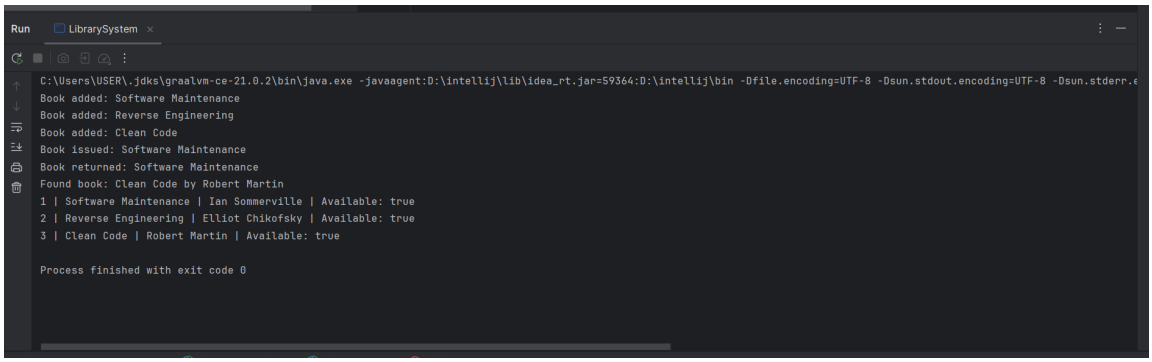


Figure 16: Output of the Java Library Management application.

4.4 Doxygen and Graphviz

Doxygen was used to recover documentation from the Java source code. Graphviz was installed so that Doxygen could create class and relationship diagrams.

The versions were verified using:

```
doxygen -v
```

```
dot -V
```

4.5 Generating the Doxyfile

PowerShell was opened in the Java project directory. The following command was used:

```
doxygen -g Doxyfile
```

This created the configuration file named `Doxyfile`.

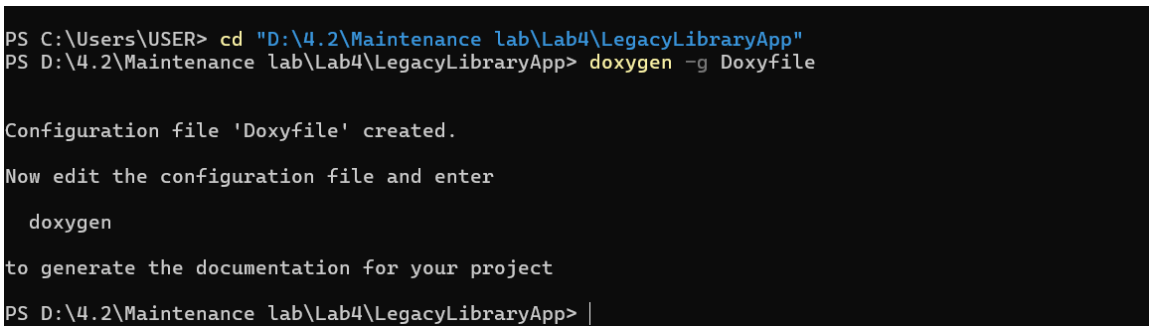


Figure 17: Doxyfile generated for the Java codebase.

4.6 Important Doxyfile Settings

Only the necessary settings were changed:

- `INPUT = src`
- `RECURSIVE = YES`
- `EXTRACT_ALL = YES`
- `OPTIMIZE_OUTPUT_JAVA = YES`
- `GENERATE_HTML = YES`
- `HAVE_DOT = YES`
- `CLASS_GRAPH = YES`
- `CALL_GRAPH = YES`
- `CALLER_GRAPH = YES`

After saving the configuration, the command below was used:

```
doxygen Doxyfile
```